

8051 Project -

SOS Button

Problem - Women's safety - prevention against

Solution - One-button alert system.

- ① when button is pressed → sends sms to some pre-defined loaded numbers
- ② ~~LCD~~ to show status of msg (sent)

Requirements → ① battery backup
② msg must be sent even if ~~network~~ is weak internet is not available.

GSM → Global system for mobile communication

↳ cellular tech. → used by phones

↳ using GSM we can -

- ① send sms
- ② receive sms
- ③ make call
- ④ receive call

Eg of GSM modules → (insert sim & use)

- ① SIM800L
- ② SIM900A

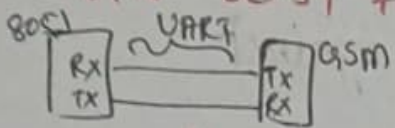
⇒ BUT it still uses 2G network → Limitation
so, 2G must be supported.

Why GSM over Wifi?

works without internet,
sends sms directly, no
need of an app.

8051 → GSM (communication)

Comm. b/w 8051 + GSM happens through UART
in-built in 8051



GSM commands used to send SMS

AT → Attention → to active GSM

AT+CMGF=1 → CMGF = cell message format

AT+CMGS="+918234167910"

EMERGENCY ALERT! HELP!

Ctrl+Z

type=1

↳ readable text

↳ the msg is stored

CMGS = cell message send → prepares GSM to send msg to given num.

↳ corresponds to AT+CMGS → used to actually send msg

Functions used

① void delay (unsigned int ms); → to provide delay

② void uart_init(); → initialises UART.

③ void uart_tx(char^{space} ch); →

In UART each character is sent serially → so this func. is declared to transmit data

④ void ~~gsm~~ cmd_to_gsm (char *str);

↳ Uses uart_tx() func. to send a string of command to gsm.

⑤ void send_sms(char * num); → to send sms to a particular number.

Main code.

```
#include <reg51.h>
```

```
sbit SOS = P1^0;
```

```
sbit BUZZER = P1^1;
```

```
void delay(unsigned int ms);
```

```
void uart_init();
```

```
void uart_tx(char ch);
```

```
void cmd_to_gsm(char * str);
```

```
void send_sms(char * num);
```

```
void main()
```

```
{
```

```
SOS = 1; → To configure make it an input pin
```

```
BUZZER = 0; → off.
```

```
uart_init(); → function call
```

```
while (1) { → always keeps checking if button is pressed
```

```
{
```

```
if (SOS == 0) → button is active low so SOS = 0 is  
Button Pressed
```

```
{
```

```
delay(50); → wait for sometime + check again if  
button is still pressed → Prevention of  
accidental press
```

```
if (SOS == 0)
```

```
{
```

```
BUZZER = 1;
```

```
send_sms("+919065608945");
```

```
send_sms(" ");
```


Buzzer = 0 ; // off.

while (SOS == 0);

↳ runs as long as button is pressed
so that 100 msgs are not sent
for 1 button press.
Why not use 'break'?

will take it out of while (1)

i.e it won't check button presses.

UART initialization

```
void uart_init()  
{  
    PCON = 0x00;  
    TMOD = 0x20;  
    TH1 = 0xFD;  
    TL1 = 0xFD;  
    SCON = 0x50;  
    TR1 = 1;  
    TI = 1;  
}
```

uart-send character

```
void uart_tx(char ch)  
{  
    TI = 0;  
    SBUF = ch; → stores  
    while (TI == 0);  
    TI = 0;  
}
```

Send string to gsm ↳ a command

```
void cmd-to-gsm(char *str)  
{  
    while (*str)  
    {  
        uart_tx(*str++);  
    }  
}
```

sms junction

```
void send_sms(char *num)
```

```
{  
    cmd_to_gsm("AT\r\n");  
    delay(1000);  
    cmd_to_gsm("AT+CMF=1\r\n");  
    delay(1000);
```

```
    cmd_to_gsm("AT+CMGS=\r\n");  
    cmd_to_gsm(num);
```

```
    cmd_to_gsm("\r\n");  
    delay(1000);
```

```
    cmd_to_gsm("EMERGENCY ALERT! HELP!\r\n");  
    uart_tx(26);  
    delay(5000);
```

```
}
```

delay junction

```
void delay(unsigned int ms)
```

```
{  
    unsigned int i, j;  
    for (i=0; i<ms; i++)
```

```
{  
    for (j=0; j<1275; j++);
```

```
}  
}
```

Additional understanding

① TMOD → Timer mode Timer 0 or Timer 1

Timer 1 → reloads timer after every count
↳ best for serial communication

TMOD Bits →

7	6	5	4	3	2	1	0
G1	C/T	M1	M0	G0	C0	M1	M0

0 0 1 0 0 0 0 0 → F01 mode

TMOD = 0x20

Importance →

- ① Sets UART Baud rate
- ② Timer operates correctly only if TMOD is set
- ③ Timer 1 → FD

FE
FE
FE
overflow
FD
FE
FF

② PCON

✓
Purpose → power saving modes

↳ baud rate doubling → we don't need this
so →

PCON = 0x00

③ SMOD → for UART speed

SMOD = 0x00 for 9600 baud rate

Registers

④ TH1 (Timer 1 High register)

→ stores reload value for timer 1
→ i.e. after overflow what value

~~EA~~ → ~~PORTS~~

For 11.0592MHz & baud rate = 9600

→ TH1 = 0xFD

⑤ TL1 (Timer 1 Low register)

→ stores current value of timer 1

TL1 = 0xFD

→ initialised to remove any garbage values.

TH1 = reload value
TL1 = current value

⑥ SCON (Serial Control register)

→ controls → UART mode
→ transmission
→ reception

SCON = 0x50

0101 0000

Mode 1 → standard UART

⑦ TI (transmit interrupt flag)

TI = 1 → means Tx complete → automatically happens.

TI = 0 → in progress

So, TI is initially set to 0

& once it becomes 1 → out of while loop → again set to 0

```
TI = 0;  
SBUF = ch;  
while (TI == 0);  
TI = 0;
```

⑧ SBUF (Serial Buffer)

- ↳ The data to send / receive is stored in SBUF
- ↳ to send a char → write to SBUF
- receive a char → receive from SBUF

Drawbacks to fix →

- ⇒ ① response time of gsm module is too slow & often one button-click is not sufficient to send the msg.
- ⇒ ② Buzzer used to indicate if msg has been sent can trigger unwanted attention.

Properties for next version

- ① Add ~~gps~~ GPS location feature
- ② Remove buzzer
- ③ Put an LED to indicate if msg has been sent / not.